Report

v. 1.0

Customer

Enso

Smart Contract Audit

Shortcuts Client Contracts

12th January 2026

Report prepared by

ABDK Consulting

Contents

1 Changelog

#	Date	Author	Description
0.1	09.01.26	A. Zveryanskaya	Initial Draft
0.2	12.01.26	A. Zveryanskaya	Minor revision
1.0	12.01.26	A. Zveryanskaya	Release

2 Summary

This document presents the results of a smart contract audit performed by ABDK Consulting for Enso's shortcuts client contracts system. The audit was conducted between 10th December and 12th January 2025 by Mikhail Vladimirov and Dmitry Khovratovich. The goal of the audit was to assess the security, correctness, and code quality of the `EnsoWalletV2` and `EnsoWalletV2Factory` contracts, which implement a factory pattern for deploying smart contract wallets with support for token transfers and execution of shortcuts.

Our conclusion is that the System demonstrates good code quality and follows established patterns from widely used libraries such as OpenZeppelin. The codebase is well-structured, utilizing the clone pattern for gas-efficient wallet deployment and integrating with Solady utilities for optimized operations. The implementation leverages inheritance from `AbstractEnsoShortcuts`, `AbstractMultiSend`, and `Withdrawable` to provide modular functionality.

We identified no critical or major vulnerabilities during our review. The most significant findings related to error handling distinguishability in both `EnsoWalletV2Factory` CVF-1 and `EnsoWalletV2` CVF-3. These issues concern the inability to distinguish between errors thrown at different levels when external calls revert without return data. While this does not affect security or correctness, it may impact off-chain diagnostics. The issues were resolved during the audit.

Overall, the contracts are production-ready following the implementation of suggested improvements to error handling.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 System overview

This section provides a high-level overview of the Enso shortcuts client contracts system, which implements a smart contract wallet infrastructure designed to enable users to deploy wallets, transfer tokens, and execute complex transaction sequences in a single operation. We were asked to review:

- Original Code
- Code with Fixes

Files:

```
/
        EnsoWalletV2.sol        EnsoWalletV2Factory.sol
```

The System facilitates the creation of user-controlled smart contract wallets through a factory pattern, allowing for efficient deployment and immediate execution of predefined actions called shortcuts.

The architecture consists of two primary components. The `EnsoWalletV2Factory` contract serves as the deployment mechanism, responsible for creating new wallet instances and coordinating initial token transfers. The `EnsoWalletV2` contract represents the wallet implementation, providing the core functionality that each deployed wallet inherits. This separation follows the minimal proxy pattern, which significantly reduces gas costs by reusing the same implementation code across multiple wallet instances.

The factory contract leverages the Solady library's `LibClone` utility for deterministic wallet deployments. This allows the System to predict wallet addresses before deployment based on the user's account address. A key feature is the `deployAndExecute` function, which accepts an array of tokens to transfer into the newly deployed wallet along with encoded calldata specifying operations to execute. This capability supports both native blockchain assets and ERC20 tokens, with built-in validation to prevent duplicate native asset transfers.

The wallet implementation inherits functionality from multiple abstract contracts. The `AbstractEnsoShortcuts` module enables execution of predefined transaction sequences. The `AbstractMultiSend` module provides transaction batching capabilities. The `Withdrawable` module handles secure fund extraction with appropriate access controls. Additionally, the System integrates OpenZeppelin's `Initializable` contract to provide proper initialization logic suitable for proxy patterns.

External integrations include interaction with the Enso Router through the `IEnsoRouter` interface, which defines the token abstraction used throughout the System. This

abstraction supports multiple token types with extensibility for additional standards. Access control operates on a per-wallet basis, where each wallet is associated with a specific user account determined during deployment. The function validates caller authorization, ensuring users maintain exclusive control over their deployed wallets while allowing the factory to perform necessary setup operations during initial deployment.

The System implements comprehensive error handling with custom error types including validation errors for incorrect transaction values, authorization errors for unauthorized callers, execution errors when called contracts revert, and duplication errors for multiple native asset transfers. These error messages facilitate debugging and provide users with actionable feedback when transactions fail.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment**. The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis**. Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis**. For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis**. The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** cover code style, best practices and general improvements.

5 Our findings

We've provided the client with a few recommendations.

Moderate	Info	Fixed
	0	2

6 Moderate Issues

CVF-1 FIXED

- **Category** Suboptimal
- **Source** EnsoWalletV2Factory.sol

Description It is impossible to distinguish the "EnsoWalletV2_ExecutionFailed" error thrown from here and the same error thrown from within the external call.

Recommendation Either always forward the error message as is, even when it is empty, or always wrap it into a named error.

Client Comment *We acknowledge the observation regarding revert distinguishability. The implementation follows the same pattern used by OpenZeppelin's low-level call helpers e.g. ['Address.verifyCallResult']: if the external call returns revert data, it is bubbled as-is; otherwise, a generic fallback error is used for the empty-revert case. The reported issue does not affect correctness, safety, or access control, and does not introduce a security vulnerability. The ambiguity only exists in the theoretical case where a callee deliberately reverts with the same custom error selector, and only affects off-chain observability of revert provenance. For this reason, we believe a Moderate severity classification overstates the impact, and consider this a low-severity / informational UX and diagnostics concern.*
That said, to remove any possible ambiguity in the empty-returndata case while preserving revert bubbling behavior, we will rename the fallback error to:

- *error EnsoWalletV2Factory_ExecutionFailedNoReason();*

and use it exclusively when the external call reverts without return data.

```
74          revert(add(0x20, response), mload(response))
```

```
77 revert EnsoWalletV2Factory_ExecutionFailed();
```

CVF-3 FIXED

- **Category** Suboptimal
- **Source** EnsoWalletV2.sol

Description It is impossible to distinguish the "EnsoWalletV2_ExecutionFailed" error thrown from here and the same error thrown from within the external call.

Recommendation Either always forward the error message as is, even when it is empty, or always wrap it into a named error.

Client Comment *We acknowledge the observation regarding revert distinguishability. The implementation follows the same pattern used by OpenZeppelin's low-level call helpers e.g. ['Address.verifyCallResult']: if the external call returns revert data, it is bubbled as-is; otherwise, a generic fallback error is used for the empty-revert case. The reported issue does not affect correctness, safety, or access control, and does not introduce a security vulnerability. The ambiguity only exists in the theoretical case where a callee deliberately reverts with the same custom error selector, and only affects off-chain observability of revert provenance. For this reason, we believe a Moderate severity classification overstates the impact, and consider this a low-severity / informational UX and diagnostics concern.*
That said, to remove any possible ambiguity in the empty-returndata case while preserving revert bubbling behavior, we will rename the fallback error to:

- *error EnsoWalletV2_ExecutionFailedNoReason();*

and use it exclusively when the external call reverts without return data.

```
50         revert(add(0x20, response), mload(response))
```

```
53 revert EnsoWalletV2_ExecutionFailed();
```

7 Recommendations

CVF-4 FIXED

- **Category** Procedural
- **Source** EnsoWalletV2Factory.sol

Recommendation Consider specifying as "^0.8.0" unless there is something special regarding this particular version. Also relevant for: EnsoWalletV2.sol.

Client Comment *We'll lower to ^0.8.20, which is the lowest version we can support. This codebase depends on dependencies like OpenZeppelin Contracts (which targets ^0.8.20) and Solady utilities (e.g., LibClone, which requires at least 0.8.4). Besides a few of our key contracts and interfaces also target ^0.8.20, and we compile the entire repository using a single compiler version for consistency and reproducibility.*

```
2 pragma solidity ^0.8.28;
```

CVF-5 INFO

- **Category** Procedural
- **Source** EnsoWalletV2Factory.sol

Description We didn't review these files.

```
4  import { Token, TokenType } from "../interfaces/IEnsoRouter.sol";
   import { IEnsoWalletV2 } from "../interfaces/IEnsoWalletV2.sol";
   import { IEnsoWalletV2Factory } from "../interfaces/
      ↪ IEnsoWalletV2Factory.sol";

10 import { LibClone } from "solady/utils/LibClone.sol";
```

CVF-6 INFO

- **Category** Procedural
- **Source** EnsoWalletV2Factory.sol

Description UPPER_CASE is conventionally used for constants, not variables.

Recommendation Use camelCase instead.

Client Comment *We disagree with this finding.*
Although UPPER_CASE is traditionally used for 'constant', it is also commonly used in practice for 'immutable' variables that are set once at deployment and never change. In this case, 'IMPLEMENTATION' represents a fixed protocol configuration value, and the naming is intentional for clarity. This is a stylistic preference and no change is required.

```
19 address public immutable IMPLEMENTATION;
```

CVF-7 INFO

- **Category** Bad datatype
- **Source** EnsoWalletV2Factory.sol

Recommendation The type for this variable should be more specific.
Client Comment *We consider this a stylistic preference rather than an issue*

```
19 address public immutable IMPLEMENTATION;
```

CVF-8 INFO

- **Category** Bad datatype
- **Source** EnsoWalletV2Factory.sol

Recommendation The argument type should be more specific.
Client Comment *We consider this a stylistic preference rather than an issue*

```
21 constructor(address implementation) {
```

CVF-9 INFO

- **Category** Bad datatype
- **Source** EnsoWalletV2Factory.sol

Recommendation The return type should be “IEnsoWalletV2”.
Client Comment *We consider this a stylistic preference rather than an issue*

```
26 function deploy(address account) external returns (address wallet) {
```

```
43 function getAddress(address account) external view returns (address)
   ↪  {
```

```
81 function _deploy(address account) private returns (address wallet) {
```

CVF-10 INFO

- **Category** Bad datatype
- **Source** EnsoWalletV2Factory.sol

Recommendation The type for the “wallet” returned value should be “IEnsoWalletV2”.
Client Comment *We consider this a stylistic preference rather than an issue*

```
37 returns (address wallet, bytes memory response)
```

```
53 returns (address wallet, bytes memory response)
```

CVF-11 INFO

- **Category** Procedural
- **Source** EnsoWalletV2.sol

Description We didn't review these files.

```
4 import { AbstractEnsoShortcuts } from "../AbstractEnsoShortcuts.sol"
    ↪ ;
  import { AbstractMultiSend } from "../AbstractMultiSend.sol";
  import { IEnsoWalletV2 } from "../interfaces/IEnsoWalletV2.sol";
  import { Withdrawable } from "../utils/Withdrawable.sol";
```

CVF-12 INFO

- **Category** Bad datatype
- **Source** EnsoWalletV2.sol

Recommendation The type for this variable should be "EnsoWalletV2Factory" or an interface derived from it.

Client Comment *We disagree with this finding.*
The factory variable is intentionally typed as address, as it is only used for sender validation and no factory interface methods are invoked. Using a more specific contract or interface type would not provide additional runtime safety and would not change behavior, since Solidity does not enforce interface compliance on addresses. The current type accurately reflects how the variable is used.

```
16 address public factory;
```

CVF-13 INFO

- **Category** Readability
- **Source** EnsoWalletV2.sol

Recommendation Should be "else revert".

Client Comment *We consider this a stylistic preference rather than an issue*

```
53 revert EnsoWalletV2_ExecutionFailed();
```

CVF-14 INFO

- **Category** Bad naming
- **Source** EnsoWalletV2.sol

Description The function name is confusing, as it is unclear what properties of “msg.sender“ are checked.

Recommendation Uee a more specific name.

Client Comment *We disagree with this finding.*
The function name ‘_checkMsgSender‘ is inherited from and overrides logic defined in the abstract base contracts (not in scope), and its purpose is to validate whether msg.sender is authorized to call the function. The specific authorization rules are intentionally defined in the function body and communicated via the ‘EnsoWalletV2_InvalidSender‘ error. Renaming the function would require changes to the inherited interfaces and would not improve correctness or clarity. As such, the current naming is intentional and appropriate.

```
61 function _checkMsgSender() internal view override(
      ↪ AbstractEnsoShortcuts, AbstractMultiSend) {
```

ABDK
Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting